Assignment 4

Semantic Search Ready NLP Pipeline from Raw Text Data

Introduction

This project focuses on building an end-to-end Natural Language Processing (NLP) pipeline capable of transforming raw, unstructured text into meaningful vector representations for semantic search. The system processes customer reviews and feedback data, cleans and standardizes the text, generates embeddings, and enables similarity-based retrieval of related comments.

The implementation uses libraries such as NLTK, spaCy, scikit-learn, Gensim, and Hugging Face Datasets to build a robust and scalable pipeline.

Dataset Description

The dataset is loaded from the Hugging Face library using the IMDB movie reviews dataset. A subset of 2000 records is used to ensure efficient processing within the computational constraints of the environment.

This dataset contains raw textual reviews, which simulate real-world customer feedback scenarios.

Step-by-Step Explanation of Implementation

1. Installation of Dependencies

```
!pip install nltk spacy scikit-learn gensim datasets matplotlib seaborn
!python -m spacy download en_core_web_sm
!pip install datasets==2.21.0
```

The first step installs all required libraries and downloads the spaCy language model.

This ensures that all tools required for text processing, vectorization, and modeling are available. The spaCy model is specifically required for linguistic processing tasks such as tokenization and lemmatization.

2. Importing Libraries

```
import re
import nltk
import spacy
import numpy as np
import pandas as pd
from datasets import load_dataset
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from gensim.models import Word2Vec
import matplotlib.pyplot as plt
import seaborn as sns
nltk.download('stopwords')
nlp = spacy.load("en_core_web_sm")
stop_words = set(stopwords.words('english'))
```

All required libraries are imported, including modules for text processing, numerical computation, dataset handling, and visualization.

This step centralizes dependencies and ensures that all subsequent operations can access the required functions without redundancy.

3. Dataset Loading

```
from datasets import load_dataset
dataset = load_dataset("imdb")
df = pd.DataFrame(dataset['train'][:2000])
df = df[['text']]
df.head()
```

The dataset is loaded using the Hugging Face dataset API and converted into a Pandas DataFrame for easier manipulation.

A subset of the dataset is selected to optimize performance and prevent excessive memory usage.

4. Text Cleaning

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"<.*?>", "", text)  # remove HTML
    text = re.sub(r"[^a-zA-Z\s]", "", text)  # remove punctuation/numbers
    text = re.sub(r"\s+", " ", text).strip()
    return text
df['clean_text'] = df['text'].apply(clean_text)
```

A custom function is implemented to clean the raw text data. This includes converting text to lowercase, removing HTML tags, eliminating special characters, and normalizing whitespace.

This step is crucial because raw text often contains noise that can negatively impact downstream processing. Cleaning ensures consistency and improves the quality of extracted features.

5. Tokenization, Stopword Removal, and Lemmatization

```
def preprocess(text):
    doc = nlp(text)
    tokens = [
        token.lemma_ for token in doc
        if token.text not in stop_words and token.is_alpha
    ]
    return tokens
df['tokens'] = df['clean_text'].apply(preprocess)
df[['clean_text', 'tokens']].head()
```

The cleaned text is processed using spaCy to perform tokenization, stopword removal, and lemmatization in a single pipeline.

Tokenization breaks text into individual words, stopword removal eliminates commonly used but insignificant words, and lemmatization converts words to their base forms.

This step standardizes the language and reduces redundancy, ensuring that semantically similar words are treated uniformly.

6. Vocabulary Creation

```
vocab = set()
for tokens in df['tokens']:
    vocab.update(tokens)
print("Vocabulary Size:", len(vocab))
```

A vocabulary is constructed by collecting all unique tokens from the processed dataset.

This vocabulary defines the feature space for vectorization techniques and ensures consistency across different representations.

Without a well-defined vocabulary, vector representations may become inconsistent or incomplete.

7. Bag of Words (BoW) Representation

```
bow_vectorizer = CountVectorizer(max_features=5000)
bow_matrix = bow_vectorizer.fit_transform(df['clean_text'])
print("BoW Shape:", bow_matrix.shape)
```

The Bag of Words model converts text into numerical vectors based on word frequency.

Each document is represented as a vector where each dimension corresponds to a word in the vocabulary. The value indicates how often the word appears in the document.

Although simple, this method provides a baseline numerical representation of text data.

8. TF-IDF Vectorization

```
tfidf_vectorizer = TfidfVectorizer(max_features=5000)
tfidf_matrix = tfidf_vectorizer.fit_transform(df['clean_text'])
print("TF-IDF Shape:", tfidf_matrix.shape)
```

TF-IDF improves upon Bag of Words by assigning weights to words based on their importance.

Words that appear frequently in a document but rarely across the dataset receive higher weights, while common words receive lower importance.

This results in a more meaningful representation of text compared to simple frequency counts.

9. Word Embeddings using Word2Vec

```
w2v_model = Word2Vec(
    sentences=df['tokens'],
    vector_size=100,
    window=5,
    min_count=2,
    workers=4
)
print("Vocabulary Size (Word2Vec):", len(w2v_model.wv))
```

A Word2Vec model is trained using the processed tokens to generate dense vector representations of words.

Unlike BoW and TF-IDF, Word2Vec captures semantic relationships between words based on their context. Words with similar meanings tend to have similar vector representations.

This step introduces semantic understanding into the pipeline.

10. Sentence Embeddings

```
def sentence_vector(tokens):
    vectors = [
        w2v_model.wv[word]
        for word in tokens if word in w2v_model.wv
    ]
    if len(vectors) == 0:
        return np.zeros(100)
    return np.mean(vectors, axis=0)
df['sentence_vector'] = df['tokens'].apply(sentence_vector)


sentence_embeddings = np.vstack(df['sentence_vector'].values)
print("Sentence Embedding Shape:", sentence_embeddings.shape)
```

Sentence embeddings are generated by averaging the vectors of individual words in a sentence.

This produces a fixed-length numerical representation for each document, which can be used for comparison and similarity analysis.

Although simple, this method is effective for many practical applications.

11. Similarity Search

```
def find_similar(query, top_n=5):
    query_clean = clean_text(query)
    query_tokens = preprocess(query_clean)
    query_vec = sentence_vector(query_tokens).reshape(1, -1)

    similarities = cosine_similarity(query_vec, sentence_embeddings)[0]
    top_indices = similarities.argsort()[-top_n:][::-1]

    results = df.iloc[top_indices]['text'].values
    return results

# Example
query = "This movie was fantastic and thrilling"
results = find_similar(query)

for i, res in enumerate(results):
    print(f"\nResult {i+1}:\n{res[:300]}...")
```

Cosine similarity is used to compare the embedding of a query with all document embeddings.

The system retrieves the most similar documents based on their vector proximity. This enables semantic search, where results are based on meaning rather than exact keyword matches.

This is the core functionality of the system and directly addresses the business requirement of finding similar comments.

Output

```python
def find_similar(query, top_n=5):
    query_clean = clean_text(query)
    query_tokens = preprocess(query_clean)
    query_vec = sentence_vector(query_tokens).reshape(1, -1)

    similarities = cosine_similarity(query_vec, sentence_embeddings)[0]
    top_indices = similarities.argsort()[-top_n:][::-1]

    results = df.iloc[top_indices]['text'].values
    return results

# Example
query = "This movie was fantastic and thrilling"
results = find_similar(query)

for i, res in enumerate(results):
    print(f"\nResult {i+1}:\n{res[:300]}...")
```

```
Result 1:
This was an incredibly stupid movie. It was possibly the worst movie I've ever had the displeasure of sitting through. I cannot fathom how it ranks a rating of 5 or 6.....

Result 2:
This was one of the worst movies i have ever seen. The plot is awful, and the acting is worse. The jokes that are attempted absolutley suck. Don't bother to waste your time on a dumb mov
ie such as this. And if for some reason that you do want to see this movie, don't watch it with your parents....

Result 3:
This has to be the worst movie I have seen. Madsen fans don't be drawn into this like I was. He is only in it for a maximum of five minutes. This movie is so bad that the only reason why
you would watch it is if all the rest of the movies on earth as well as t.v. had been destroyed....

Result 4:
Really, I can't believe that I spent $5 on this movie. I am a huge zombie fanatic and thought the movie couldn't be that bad. It had zombies in it right? Was I wrong! To be honest the mo
vie had it's moments...I thought it was cool when the guy got his head ripped off but that was about it. Overall I...

Result 5:
This is definitely the worst movie Adam's ever done but at this point in his life, he was just happy to have a movie. There are 3 or 4 laughs in it but I used the fast forward button thr
ough some of it. Don't waste your time. I only saw it because I wanted to see all of his movies, but it sucked....
```

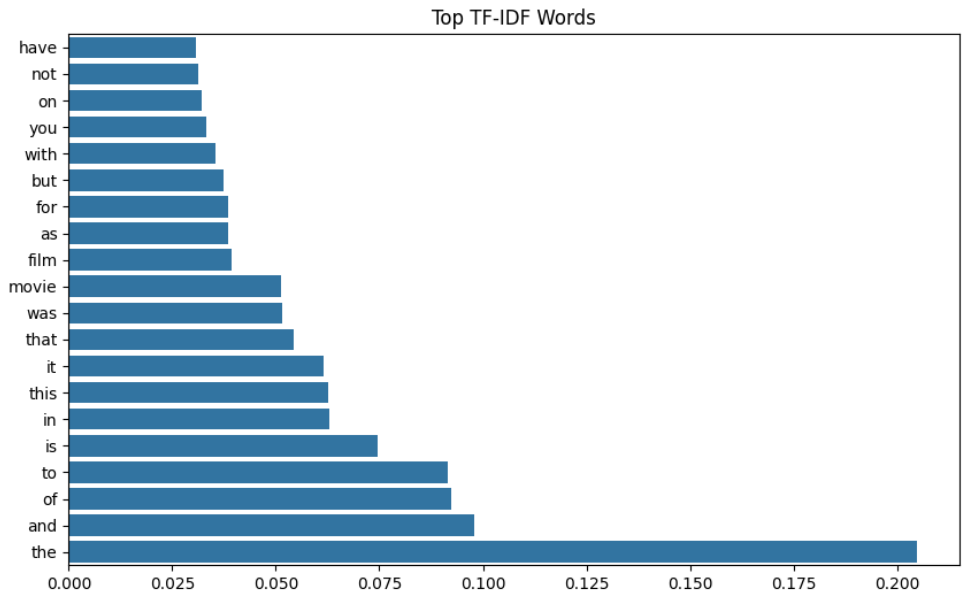
12. Visualization

```python
feature_names = tfidf_vectorizer.get_feature_names_out()
scores = np.mean(tfidf_matrix.toarray(), axis=0)

top_indices = scores.argsort()[-20:]

plt.figure(figsize=(10,6))
sns.barplot(x=scores[top_indices], y=feature_names[top_indices])
plt.title("Top TF-IDF Words")
plt.show()
```

A visualization of the top TF-IDF words is created using plotting libraries.

This helps in understanding which terms are most significant in the dataset and provides insights into the dominant themes in the text.

Visualization is important for interpretability and communicating findings to non-technical stakeholders.



13. Insights and Analysis

```
avg_length = df['tokens'].apply(len).mean()


print("Average Tokens per Document:", avg_length)


# Most common words
from collections import Counter


all_words = [word for tokens in df['tokens'] for word in tokens]
common_words = Counter(all_words).most_common(10)


print("\nTop 10 Words:")
for word, freq in common_words:
    print(word, freq)
```

Basic statistical analysis is performed, including calculating the average number of tokens per document and identifying the most frequent words.

These insights help in understanding the structure and characteristics of the dataset, which can guide further optimization and model improvements.

Output

11:27 AM (<1s) 14

```python
avg_length = df['tokens'].apply(len).mean()

print("Average Tokens per Document:", avg_length)

# Most common words
from collections import Counter

all_words = [word for tokens in df['tokens'] for word in tokens]
common_words = Counter(all_words).most_common(10)

print("\nTop 10 Words:")
for word, freq in common_words:
    print(word, freq)
```

```
Average Tokens per Document: 114.4985

Top 10 Words:
movie 4266
film 3457
not 3015
one 2023
make 1875
like 1782
see 1724
bad 1599
get 1472
good 1357
```